Implementacja algorytmu mnożenia liczb dowolnej precyzji

Cybulski Mateusz, Derron Tomasz, Domiński Bartosz

Streszczenie

Implementacja algorytmu mnożenia Toom-Cook'a (Toom-3) dla liczb całkowitych dowolnej precyzji w C++. Zaprojektowanie i przeprowadzenie eksperymentu mającego na celu porównanie utworzonej implementacji z istniejącymi bibliotekami: GMP i wbudowaną biblioteką w Python.

Spis treści

1 Metodologia **2**
1.1 Cel eksperymentu . . . 2
1.2 Opis eksperymentu . . . 2
1.3 Plan osiągnięcia celów projektowych . . . 2

2 Własna implementacja rozwiązania **4**
2.1 Rola sztucznej inteligencji w procesie projektowania struktury . . . 4
2.2 Decyzje architektoniczne . . . 4
2.3 Analiza pierwotnej wydajności . . . 4
2.4 Optymalizacja i refaktoryzacja . . . 5
2.5 Wyniki po optymalizacji . . . 5
2.6 Dopasowanie do modelu teoretycznego . . . 5

3 Implementacja z wykorzystaniem biblioteki GMP **6**

4 Implementacja w języku Python **7**

5 Dodatek **8**
5.1 Dokumentacja promptów . . . 8
5.2 Jak duże liczby można pomnożyć w ciągu sekundy? . . . 10
5.2.1 Generowanie danych testowych . . . 10
5.2.2 Wyniki eksperymentu . . . 10
5.3 Dokumentacja techniczna . . . 12
5.3.1 Generowanie próbki . . . 12
5.3.2 Implementacja w języku Python . . . 12
5.3.3 Implementacja z wykorzystaniem biblioteki GNU MP . . . 12
5.3.4 Analiza wyników . . . 13

1 Metodologia

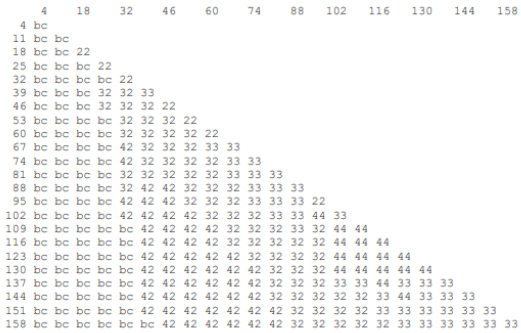
1.1 Cel eksperymentu

Celem eksperymentu jest zbadanie zależności pomiędzy czasem wykonania operacji mnożenia a rozmiarem danych wejściowych dla utworzonej implementacji algorytmu mnożenia dużych liczb. W drugiej części eksperymentu, na podstawie otrzymanej zależności, nastąpi porównanie naszej implementacji z istniejącymi bibliotekami.

1.2 Opis eksperymentu

Zbiór liczb użyty w eksperymencie będzie składał się z liczb w zapisie o podstawie $\beta = 2^{32}$. W eksperymencie zostaną użyte liczby z zakresu od liczb 50-słowowych do 1000-słowowych (zakładamy, że słowo = 32 bity).

Będziemy mierzyć czas wykonania operacji mnożenia dla liczb o długości 50, 100, 150, ... , 950, 1000 słów. Dla każdej z podanych długości zostanie wygenerowanych 40 liczb o zadanej długości, następnie w parach (20 par dla każdej z ustalonych długości) zostanie przeprowadzone mnożenie. Zakres długości liczb został wybrany na podstawie wykresu 1, pokazanym poniżej.



	4	18	32	46	60	74	88	102	116	130	144	158
4	bc											
11	bc	bc										
18	bc	bc	22									
25	bc	bc	bc	22								
32	bc	bc	bc	bc	22							
39	bc	bc	bc	32	32	33						
46	bc	bc	bc	32	32	32	22					
53	bc	bc	bc	bc	32	32	32	22				
60	bc	bc	bc	bc	32	32	32	32	22			
67	bc	bc	bc	bc	42	32	32	32	33	33		
74	bc	bc	bc	bc	42	32	32	32	32	33	33	
81	bc	bc	bc	bc	32	32	32	32	32	33	33	33
88	bc	bc	bc	bc	32	42	42	32	32	32	33	33	33
95	bc	bc	bc	bc	42	42	42	32	32	32	33	33	33	22
102	bc	bc	bc	bc	42	42	42	42	32	32	32	33	33	44	33
109	bc	bc	bc	bc	bc	42	42	42	42	32	32	32	33	32	44	44
116	bc	bc	bc	bc	bc	bc	42	42	42	42	32	32	32	32	32	44	44	44
123	bc	bc	bc	bc	bc	bc	42	42	42	42	42	32	32	32	32	44	44	44	44
130	bc	bc	bc	bc	bc	bc	42	42	42	42	42	42	32	32	32	44	44	44	44	44
137	bc	bc	bc	bc	bc	bc	42	42	42	42	42	42	32	32	32	33	33	44	33	33	33
144	bc	bc	bc	bc	bc	bc	42	42	42	42	42	42	32	32	32	32	32	33	44	33	33	33
151	bc	bc	bc	bc	bc	bc	42	42	42	42	42	42	42	32	32	32	32	33	33	33	33	33	33
158	bc	bc	bc	bc	bc	bc	bc	42	42	42	42	42	42	32	32	32	32	32	33	33	33	33	33	33

Figure 1.1 The best algorithm to multiply two numbers of x and y words for $4 \le x \le y \le 158$: bc is schoolbook multiplication, 22 is Karatsuba's algorithm, 33 is Toom-3, 32 is Toom-(3, 2), 44 is Toom-4, and 42 is Toom-(4, 2). This graph was obtained on a Core 2, with GMP 5.0.0, and GCC 4.4.2. Note that for $x \le (y+3)/4$, only the schoolbook multiplication is available; since we did not consider the algorithm that cuts the larger operand into several pieces, this explains why bc is best for say $x = 32$ and $y = 158$.

Rysunek 1: Algorytm mnożenia stosowany w zależności od długości słów. *Modern Computer Arithmetic*

Następnie zostanie zbadana zależoność czasu potrzebnego do przeprowadzenia operacji mnożenia w zależności od tego jak duże są mnożone liczby dla utworzonej przez nas implementacji.

Następnie te same pary liczb zostaną pomnożone z wykorzystaniem biblioteki GMP (GNU Multiple Precision Arithmetic Library) oraz w języku Python. Umożliwi to wyznaczenie analogicznej zależności jak dla naszej implementacji, oraz ich porównanie.

1.3 Plan osiągnięcia celów projektowych

Szkielet implementacji

- Przygotowanie specyfikacji – opis struktury programu, reprezentacji liczb BigInt oraz wymagań dla algorytmu Toom-3.
- Generowanie wstępnego szkieletu kodu – wykorzystanie modelu Gemini 3 do wygenerowania podstawowej struktury projektu w C++.
- Dokumentacja promptów – zapisywanie wszystkich użytych promptów oraz otrzymanych odpowiedzi (prompt documentation).
- Weryfikacja wygenerowanego kodu – ręczna weryfikacja oraz przygotowanie do dalszej implementacji algorytmów.

Implementacja dodawania BigInt

- Algorytm dodawania – niezbędny do poprawnego działania mnożenia.

Algorytm mnożenia

- Implementacja Toom-3 – wykorzystując wcześniejszy szkielet.

Eksperyment i benchmarki

- Przygotowanie próbek – z użyciem gotowej biblioteki do generowania liczb pseudolosowych i zapis danych wejściowych według ustalonej metodologii.
- Testy własnej implementacji – pomiar czasu mnożenia próbek.
- Testy rozwiązań GMP – benchmark względem profesjonalnego standardu C++.
- Testy rozwiązań wbudowanych w Pythonie – pomiar czasu w Pythonie.
- Zapisanie wyników – względem ustalonej metodologii (.csv).

Analiza wyników

- Wykresy i analiza danych – wykorzystanie bibliotek takich jak scipy, numpy, matplotlib, seaborn, pandas.
- Wnioski końcowe – podsumowanie efektywności algorytmu w praktyce.

2 Własna implementacja rozwiązania

2.1 Rola sztucznej inteligencji w procesie projektowania struktury

W początkowej fazie prac nad implementacją algorytmu Toom-Cook 3 (Toom-3) zdecydowano się na wykorzystanie modelu językowego Google Gemini 1.5 Pro, zoptymalizowanego pod kątem zadań programistycznych oraz zaawansowanych zagadnień matematycznych.

Rola modelu obejmowała wygenerowanie szkieletu struktury w języku C++ dla liczb całkowitych o dowolnej precyzji (arbitrary-precision arithmetic) oraz implementację samych algorytmów na podstawie dostarczonych założeń architektonicznych. Dokumentacja interakcji z modelem językowym została zawarta w rozdziale 5.1.

2.2 Decyzje architektoniczne

Zaprojektowana struktura opiera się na kontenerze `std::vector` z biblioteki standardowej C++, przechowującym elementy typu **`uint32_t`** (32-bitowe liczby całkowite bez znaku). W pierwotnej wersji przyjęto konwencję przechowywania słów w kolejności od najbardziej znaczącego (*Most Significant Word* – MSW) pod indeksem zerowym, co odpowiada naturalnemu zapisowi pozycyjnemu liczb.

2.3 Analiza pierwotnej wydajności

Model AI wykazał się wysoką sprawnością w odwzorowaniu logicznej struktury algorytmu, poprawnie implementując rekurencję oraz operacje na współczynnikach. Jednakże testy wydajnościowe przeprowadzone na wygenerowanym kodzie dały rezultaty znacząco odbiegające od oczekiwań oraz wydajności standardowych rozwiązań, takich jak biblioteka GMP czy wbudowane mechanizmy języka Python.

Implementacja	Czas wykonania
Biblioteka Python (wbudowana)	$\approx 470\ \mu$s
Pierwotna implementacja C++ (AI)	≈ 48 ms

Tabela 1: Porównanie czasu wykonania mnożenia (liczby o długości 1000 słów 32-bitowych)

Analiza wykazała, że implementacja była blisko 100-krotnie wolniejsza od interpretera języka Python, co w przypadku języka kompilowanego jest wynikiem anomalnym. Zidentyfikowano dwa główne czynniki obniżające wydajność:

Nadmiarowe kopiowanie obiektów Wstępny kod przekazywał parametry funkcji przez wartość zamiast przez referencję do stałej (`const &`). Brak kwalifikatora `const` oraz referencji wymuszał na programie tworzenie kopii wektorów przy każdym wywołaniu rekurencyjnym. Błąd ten występował już na poziomie konstruktora, który wykorzystywany jest w algorytmie wielokrotnie:

```
// BŁĄD: Niepotrzebne kopiowanie całego wektora
WielkaLiczba(std::vector<uint32_t> liczba) {
    slowa = liczba;
}
```

Wprowadzenie przekazywania przez referencję pozwoliło na znaczną redukcję narzutu czasowego (Tabela 2).

Wersja konstruktora	Czas wykonania
Bez referencji i const	≈ 29 ms
Z referencją i const	≈ 16 ms

Tabela 2: Czas utworzenia 100 000 obiektów struktury `WielkaLiczba`

Nieefektywna reprezentacja Big-Endian Przechowywanie słów w kolejności od MSW do LSW (Big-Endian) wymuszało wstawianie nowych elementów (np. przeniesień) na początek wektora. Operacja `std::vector::insert(begin(), ...)` posiada złożoność liniową $O(n)$, co wewnątrz pętli algorytmów mnożenia prowadziło do degradacji złożoności całego algorytmu do poziomu kwadratowego względem liczby słów.

Reprezentacja	Liczba operacji	Czas wykonania
Big-Endian (insert na początku)	100 000	$\approx 477,1$ ms
Little-Endian (push_back)	100 000	$\approx 3,8$ ms

Tabela 3: Porównanie wydajności dopisywania elementów w zależności od reprezentacji

2.4 Optymalizacja i refaktoryzacja

Podjęto decyzję o zmianie wewnętrznej reprezentacji liczb na *Least Significant Word first* (LSW-first / Little-Endian). Przeprowadzono refaktoryzację kodu, zastępując operacje wstawiania na początku efektywnymi dopisaniami na końcu wektora (`push_back`), charakteryzującymi się złożonością stałą $O(1)$.

Dane wejściowe są nadal wczytywane w formacie Big-Endian, jednak ich kolejność jest odwracana na etapie inicjalizacji, co nie wpływa na mierzony czas operacji arytmetycznych.

2.5 Wyniki po optymalizacji

Zmiana architektury danych pozwoliła na zwiększenie wydajności o rzędy wielkości.

Implementacja	Czas wykonania
Biblioteka Python (wbudowana)	≈ 470 μs
Zoptymalizowana implementacja C++	≈ 8 ms

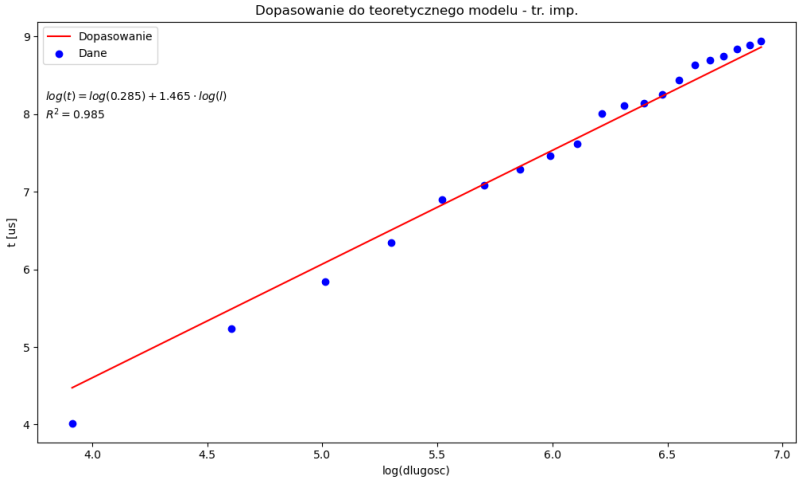
Tabela 4: Porównanie czasu wykonania po optymalizacji (1000 słów 32-bitowych)

Pozostała różnica względem rozwiązań profesjonalnych (np. GMP) wynika z braku niskopoziomowych optymalizacji, takich jak:

- **Niewykorzystanie pełnej szerokości rejestrów:** Zastosowanie słów 32-bitowych na architekturze 64-bitowej nie wykorzystuje w pełni mocy obliczeniowej procesora. Biblioteki klasy GMP operują na 64-bitowych jednostkach (*limbs*) [3], co dwukrotnie redukuje liczbę wymaganych operacji elementarnych.
- **Zarządzanie pamięcią:** `std::vector` dokonuje częstych reallokacji. Profesjonalne biblioteki stosują *pre-allocation*, rezerwując pamięć na wynik operacji z góry.

2.6 Dopasowanie do modelu teoretycznego

Po przekształceniu uzyskanych wyników dopasowano do nich linię regresji przedstawioną na poniższym wykresie. Otrzymana wartość współczynnika determinacji R^2 świadczy o dobrym dopasowaniu.



Rysunek 2: Dopasowanie danych do modelu o teoretycznej złożoności asymptotycznej $O(n^{1,465})$

.

3 Implementacja z wykorzystaniem biblioteki GMP

W celu uzyskania punktu odniesienia dla własnego rozwiązania, zaimplementowano moduł wykorzystujący bibliotekę GMP (*GNU Multiple Precision Arithmetic Library*). Na podstawie dokumentacji technicznej [3] opracowano algorytm konwertujący wejściowe kontenery `std::vector` na obiekty typu `mpz_t`. Same operacje arytmetyczne zostały wykonane bezpośrednio przez wewnętrzne, silnie zoptymalizowane procedury biblioteki, bez ingerencji w strukturę algorytmu mnożenia.

Program wczytuje dane wejściowe w formacie MSW-first (kolejność od najbardziej znaczącego słowa). W porównaniu do własnej implementacji, sposób ten nie wpływa jednak na miarodajność testów, ponieważ biblioteka GMP po zaimportowaniu danych przechowuje je zgodnie z własną strukturą.

Należy podkreślić, że operacje inicjalizacji zmiennych typu `mpz_t`, importu danych (funkcja `mpz_import`) oraz zwalniania pamięci (`mpz_clear`) nie wpływają na prezentowane wyniki wydajnościowe. Licznik czasu obejmuje wyłącznie egzekucję samej operacji mnożenia dwóch liczb.

Wybór biblioteki GMP jako punktu odniesienia podyktowany był jej statusem standardu przemysłowego w zakresie obliczeń o dowolnej precyzji. Jest ona wykorzystywana w systemach algebry komputerowej (np. Mathematica, Maple) oraz bibliotekach kryptograficznych [1]. GMP charakteryzuje się:

- **Wysoką optymalizacją niskopoziomową:** Kluczowe pętle obliczeniowe są napisane w asemblerze, specyficznym dla danej architektury procesora CPU.
- **Zaawansowanymi algorytmami:** Biblioteka automatycznie dobiera algorytm mnożenia (np. Karacuba, Toom-Cook, FFT) w zależności od rozmiaru danych wejściowych, oraz samego procesora, co czyni ją niezwykle trudną do prześcignięcia przez generyczne implementacje w językach wysokiego poziomu.

Zastosowanie GMP pozwala zatem w obiektywny sposób ocenić szybkosć innych implementacji, jako rozwiązanie o maksymalnej możliwej wydajności.

4 Implementacja w języku Python

Kolejnym etapem prac było przygotowanie skryptu w języku Python, który posłużył jako referencyjny punkt odniesienia dla wydajności mnożenia. Wybór tego języka był podyktowany faktem, iż Python posiada dynamiczną obsługę liczb całkowitych o dowolnej precyzji, która jest silnie zoptymalizowana i zaimplementowana bezpośrednio w jądrze interpretera (napisanym w języku C).

Metodyka pomiaru i konwersja danych W celu zapewnienia rzetelności porównania, przygotowano środowisko testowe spełniające warunki pracy implementacji w C++. warunki dla mierzenia czasu egzekucji mnożenia to:

- **Transparentność formatu danych:** Dane wejściowe, przechowywane w formacie MSW-first (analogicznie do pierwotnych założeń projektu), są konwertowane na wewnętrzny format typu `int` Pythona za pomocą operacji przesunięć bitowych. Pozwala to na zachowanie spójności zestawów testowych między różnymi implementacjami.
- **Izolacja operacji:** Procedura pomiarowa obejmuje wyłącznie samą operację mnożenia, z pominięciem czasu wczytywania danych oraz ich konwersji, co pozwala na bezpośrednie porównanie czystej wydajności algorytmicznej.

Wybór Pythona Wykorzystanie Pythona jako benchmarku pozwala na ocenę, jak blisko poziomu optymalizacji gotowych systemów znajduje się własna implementacja C++. Stanowi to istotny etap weryfikacji, gdyż pozwala odróżnić błędy projektowe (takie jak nieefektywne trzymanie danych, o których wspomniano w rozdziale 2.3) od różnic, na które są naturalne dla początkujących projektów.

5 Dodatek

5.1 Dokumentacja promptów

Do wygenerowania szkieletu kodu oraz implementacji algorytmów wykorzystano model językowy **Google Gemini 1.5 Pro**. Poniżej zestawiono prompty, które posłużyły do opracowania finalnego rozwiązania.

Prompt 1: Architektura i Szkielet Systemu

Cel: Stworzenie fundamentów struktury `WielkaLiczba` w języku C++ przy założeniu architektury.

Prompt Inicjujący

Zaprojektuj w C++ strukturę `WielkaLiczba` do obsługi arytmetyki o dowolnej precyzji (arbitrary-precision arithmetic).

- Użyj `std::vector<uint32_t>` jako wewnętrznego kontenera na słowa 32-bitowe.
- Przyjmij konwencję **Big-Endian** (Most Significant Word pod indeksem zero).
- Dodaj pole `bool czy_ujemna`.
- Przygotuj szkielet dla algorytmu mnożenia **Toom-Cook 3 (Toom-3)** oraz niezbędne funkcje pomocnicze (porównywanie, wczytywanie z pliku, proste dodawanie i odejmowanie).

Prompt 2: Implementacja Logiki Algorytmu Toom-3

Cel: Odwzorowanie logicznej struktury algorytmu, w tym rekurencji oraz operacji na współczynnikach.

Prompt Implementacyjny

Zaimplementuj algorytm mnożenia **Toom-Cook 3** dla klasy `WielkaLiczba`.

- Algorytm powinien dzielić liczbę na 3 części o długości $k = \lceil n/3 \rceil$.
- Zaimplementuj wyznaczanie punktów $v_0, v_1, v_{-1}, v_2, v_\infty$.
- Dodaj funkcję `div_exact` do dzieleń przez małe stałe (np. przez 2 i 6) występujących w fazie interpolacji.
- Pamiętaj o obsłudze znaków przy wyznaczaniu punktów ujemnych.
- W przypadku małych liczb, przełączaj się na algorytm Karatsuby lub mnożenie pisemne (progowe wartości N_0 i N_1).

Prompt 3: Optymalizacja i Refaktoryzacja

Cel: Eliminacja błędów wydajnościowych zidentyfikowanych podczas testów (nadmiarowe kopiowanie, nieefektywna reprezentacja Big-Endian).

Prompt Optymalizacyjny

Mój obecny kod klasy `WielkaLiczba` wykazuje niską wydajność. Przeprowadź refaktoryzację w celu optymalizacji:

- **Zmiana reprezentacji:** Przejdź na format **Little-Endian** (LSW first), aby zastąpić operację `insert(begin(), ...)` efektywnym `push_back`.
- **Eliminacja kopiowania:** Zmień przekazywanie parametrów z wartości na referencję do stałej na przykłąd: (`const WielkaLiczba&`) w celu uniknięcia kopiowania wektorów przy rekurencji.

5.2 Jak duże liczby można pomnożyć w ciągu sekundy?

W ramach rozszerzenia projektu podjęto próbę wyznaczenia granic wydajnościowych współczesnych rozwiązań w zakresie arytmetyki komputerowej. Głównym celem tej części badania było określenie rzędu wielkości liczb (wyrażonego w liczbie słów maszynowych), których operacja mnożenia mieści się w zadanym przybliżeniu czasowym: 1 milisekundy oraz 1 sekundy.

Do przeprowadzenia testów wykorzystano bibliotekę GNU MP (GMP), z racji najefektywniejszej implementacji. Należy zaznaczyć, że wyniki są ściśle związane z architekturą sprzętową komputera, na którym przeprowadzony jest eksperyment. Pomiary przeprowadzono na jednostce wyposażonej w procesor Intel Core i5-7400 3.0 GHz.

Na podstawie wstępnej empirycznej analizy ustalono, że:

- Granica czasu wykonania wynosząca 1 ms odpowiada liczbom o długości od 8 700 do 9 000 limbów (słów 32-bitowych).
- Granica czasu wykonania wynosząca 1 s odpowiada liczbom o długości od 3 650 000 do 3 700 000 limbów.

5.2.1 Generowanie danych testowych

W celu weryfikacji postawionych hipotez opracowano skrypt w języku Python, wykorzystujący mechanizmy opisane w rozdziale 5.3.1. Wygenerowano dwa zestawy danych testowych:

1. 7 par liczb o długości od 8 700 do 9 000 limbów z krokiem wynoszącym 50 słów.
2. 6 par liczb o długości od 3 650 000 do 3 700 000 limbów z krokiem wynoszącym 10 000 słów.

5.2.2 Wyniki eksperymentu

Dane poddano operacji mnożenia z wykorzystaniem implementacji opartej na bibliotece GMP (opisanej w rozdziale 5.3.3). Uzyskane wyniki prezentują się następująco:

Liczba słów	**Czas wykonania**	**Jednostka**
8 700	1,02190	ms
8 750	1,06987	ms
8 800	1,03674	ms
8 850	984,155	μs
8 900	972,745	μs
8 950	1,01806	ms
9 000	987,652	μs
3 650 000	977,098	ms
3 660 000	990,404	ms
3 670 000	1,04706	s
3 680 000	1,05475	s
3 690 000	1,04270	s
3 700 000	1,02102	s

Tabela 5: Wyniki pomiarów czasu mnożenia dla wybranych rzędów wielkości.

Aby zobrazować skalę problemu obliczeniowego, wyznaczono teoretyczną liczbę cyfr dziesiętnych dla rekordu o długości 3 660 000 słów maszynowych. Przyjmując, że każde słowo składa się z 32 bitów, całkowita liczba bitów B wynosi:

$$B = 3\,660\,000 \times 32 = 117\,120\,000 \text{ bitów}$$

Liczbę cyfr dziesiętnych D wyznacza się, korzystając z własności logarytmów. Dla liczby x, liczba jej cyfr w systemie o podstawie 10 jest równa $\lfloor \log_{10}(x) \rfloor + 1$. W przybliżeniu można przyjąć zależność:

$$\begin{aligned} D &\approx B \times \log_{10}(2) \\ D &\approx 117\,120\,000 \times 0,301029995 \\ D &\approx 35\,256\,633 \end{aligned}$$

Z powyższych obliczeń wynika, że badana liczba posiada ponad **35 milionów cyfr dziesiętnych**.

Przyjmując założenie o szerokości pojedynczej cyfry równej 0,5 cm, zapisanie tej wartości w jednej linii wymagałoby płaszczyzny o długości:

$$L = 35\,256\,633 \times 0,005 \text{ m} \approx 176\,283 \text{ m} \approx 176,3 \text{ km}$$

Tyle ile wynosi mniej więcej odległośc między Gdańskiem a Toruniem.

Fakt, że biblioteka GMP jest w stanie wykonać operację mnożenia dwóch tak ogromnych obiektów w czasie poniżej jednej sekundy ($990,404$ ms), świadczy o niezwykle wysokiej efektywności zaimplementowanych algorytmów (prawdopodobnie Szybkiej Transformaty Fouriera - FFT) oraz współczesnych jednostek CPU.

5.3 Dokumentacja techniczna

5.3.1 Generowanie próbki

Próbka jest wygenerowana z użyciem języka Python, konkretnie za pomocą modułu **random** (patrz: *random — Generate pseudo-random numbers* [6]).

Zbiory są wygenerowane jak wspomniano w metodologii. Uruchomiony skrypt: **data_gen.py** tworzy 20 plików tekstowych, każdy przeznaczony liczbom o danej wielkości (ilości słów).

Każda z stu linii jest przeznaczona na jedną liczbę. Są one zapisane w bazie $\beta = 2^{32}$. Każde słowo maszynowe jest rozdzielone znakiem spacji i są one ułożone w kolejności od najbardziej znaczącej do najmniej znaczącej.

Skrypt generuje po kolei każde słowo za pomocą metody **.getrandbits(k)**, gdzie nasze $k = 32$. Metoda ta korzysta głównie z generatora Mersenne'a Twister'a, który generuje liczby pseudolosowe o rozkładzie jednostajnym (patrz: "Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudorandom Number Generator" [4]).

5.3.2 Implementacja w języku Python

Język Python posiada natywną obsługę liczb całkowitych o dowolnej precyzji (*arbitrary-precision integers*), co eliminuje konieczność stosowania zewnętrznych bibliotek. Rozmiar reprezentowanych liczb nie posiada sztywnego ograniczenia technicznego [5]; jedynym czynnikiem limitującym jest dostępna pamięć operacyjna komputera.

Skrypt **main_PYTHON.py** składa się z zestawu funkcji pomocniczych oraz funkcji głównej *main_func*, odpowiedzialnej za przeprowadzenie operacji mnożenia oraz pomiar czasu ich wykonania.

vector_to_int Funkcja pomocnicza służąca do konwersji wektora 32-bitowych słów maszynowych na typ `int` języka Python. Jako argument przyjmuje wektor słów, a zwraca obiekt o nieograniczonej precyzji. Proces rekonstrukcji liczby opiera się na przesunięciu bitowym w lewo (*bit-shift*) oraz sumowaniu kolejnych fragmentów danych przy użyciu operatora alternatywy bitowej (*OR*).

choose_time_unit Funkcja odpowiedzialna za automatyczny dobór jednostki czasu oraz odpowiednie skalowanie wartości liczbowej w celu poprawy czytelności wyników. Przyjmuje czas trwania operacji w mikrosekundach i zwraca krotkę *(wartość, jednostka)*, gdzie wartość jest typu `float`, a jednostka typu `string`.

main_func Główna procedura sterująca procesem obliczeniowym. Realizuje ona mnożenie par liczb wczytanych z pliku, a następnie zapisuje czas egzekucji w pliku wynikowym. Funkcja przyjmuje dwie ścieżki do plików: wejściową oraz wyjściową.

Zgodnie z przyjętą metodologią, plik wejściowy musi zawierać parzystą liczbę rekordów (po jednym w każdym wierszu), reprezentujących liczby o podstawie $\beta = 2^{32}$. Funkcja wczytuje dane parami, dokonuje ich konwersji na typ `int`, a po wykonaniu mnożenia i sformatowaniu jednostek czasu, zapisuje wynik w formacie:

liczba słów maszynowych, czas (z precyzją do dwóch miejsc po przecinku), jednostka

5.3.3 Implementacja z wykorzystaniem biblioteki GNU MP

Biblioteka GNU MP (GMP) jest wysokowydajnym narzędziem napisanym w języku C, przeznaczonym do wykonywania operacji arytmetycznych na liczbach o dowolnej precyzji. W niniejszym projekcie wykorzystano wyłącznie moduł operacji na liczbach całkowitych (`mpz_t`). Wykorzystana implementacja pochodzi z oficjalnej dystrybucji i nie została poddana modyfikacjom.

vector_to_mpz Funkcja pomocnicza dokonująca transformacji wektora typu `std::vector<uint32_t>` na obiekt struktury `mpz_t`. Jej zadaniem jest ułatwienie procesu importu danych i poprawa czytelności kodu źródłowego. Funkcja nie zwraca wartości (`void`), lecz modyfikuje przekazany przez referencję obiekt `mpz_t` za pomocą funkcji `mpz_import`, zgodnie ze specyfikacją dokumentacji GMP[3].

TimeResult W celu ustrukturyzowania danych pomiarowych zdefiniowano strukturę `TimeResult`, zawierającą dwa pola: `double value` (wartość liczbowa) oraz `std::string unit` (oznaczenie jednostki). Ułatwia ona zarządzanie wynikami pomiarów i ich późniejszą prezentację.

choose_time_unit Funkcja analogiczna do wersji Pythonowskiej, dokonująca konwersji surowego pomiaru czasu (wyrażonego w nanosekundach) na obiekt struktury `TimeResult`.

main_func Główna funkcja odpowiedzialna za wczytywanie danych, realizację mnożenia oraz zapisanie wyników biblioteki GMP. Przyjmuje nazwy plików wejściowego i wyjściowego.

Podobnie jak w przypadku skryptu Python, wymagany jest plik z parzystą liczbą wierszy zawierających liczby o podstawie $\beta = 2^{32}$. Funkcja importuje dane do formatu `mpz_t`, wykonuje mnożenie, a następnie zapisuje parametry eksperymentu do pliku wyjściowego w formacie identycznym jak w przypadku implementacji w Pythonie.

Funkcja zwraca kod błędu `1` w przypadku niepowodzenia dostępu do plików lub `0` po pomyślnym zakończeniu wszystkich procedur obliczeniowych.

5.3.4 Analiza wyników

Zebrane dane zostały poddane analizie w celu zbadania zależności czasu potrzebnego na wykonanie mnożenia od rozmiaru mnożonych liczb oraz użytego narzędzia. Do uruchomienia wszystkich napisanych skryptów w odpowiedniej kolejności użyto narzędzia CMake. Za przygotowanie do analizy oraz samą analizę otrzymanych wyników odpowiada skrypt napisany w języku Python o nazwie **analysis.py**. Zawiera on funkcje opisane poniżej.

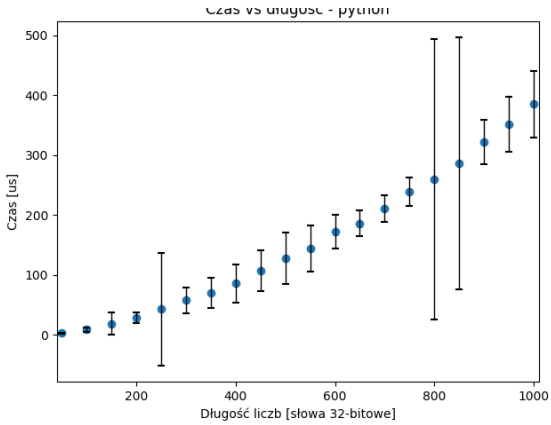
load_file Funkcja pobiera zawartość wybranego pliku ze zmierzonymi czasami potrzebnymi na wykonanie operacji mnożenia. Funkcja zwraca obiekt ramki danych zawierający dane przeznaczone do analizy. Funkcja ta zgłasza błąd w przypadku gdy szukany plik nie istnieje.

equalize_units Funkcja ujednolica jednostki, w których zapisane są czasy w wybranej ramce danych.

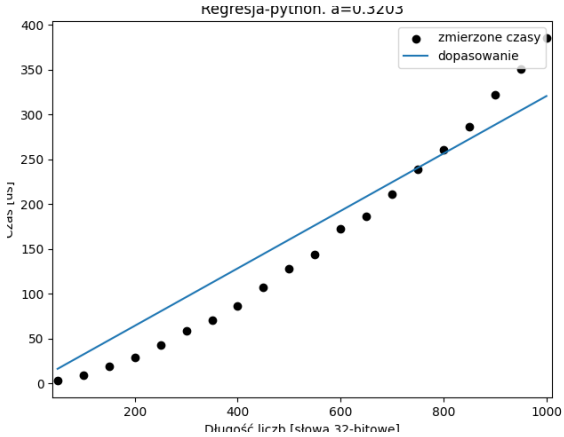
remove_outliers Funkcja ta usuwa wartości odstające w zbiorze czasów uzyskanych dla liczb o różnych długościach. Reguła, którą zastosowano do odrzucenia wartości odstających to tzw. reguła trzech σ, polegająca na odrzucenia wartości różniących się o więcej niż trzy odchylenia standardowe od średniej.

calculate_stats Funkcja ta liczy czas średni oraz odchylenie standardowe w zbiorze otrzymanych czasów dla liczb o różnej długości.

plot_one_method Funkcja ta generuje wykres dla wybranej metody. Oś pozioma przedstawia rozmiar liczb w słowach 32-bitowych, a oś pionowa czas w mikrosekundach. Średnie czasy przedstawione są za pomocą punktów. Odchylenia sytandardowe przedsatwione są za pomocą punktów. Przykładowy wykres dla mnożenia wykonanego w Pythonie został przedstawiony poniżej.



draw_regression Funkcja ta dopasowuję regresję nieliniową o równaniu zdefiniowanym w funkcji **model** do uzyskanych czasów. W kolejnym kroku generowany jest odpowiedni wykres. Przykładowy wykres dla mnożenia wykonanego w Pythonie został przedstawiony poniżej, tutaj dla przypadku gdy dopasowywana funkcja jest funkcją liniową.



workflow Funkcja ta odpowiada za wywołanie funkcji przedstawionych powyżej w odpowiedniej kolejności.

References

[1] Autorzy Wikipedii. *GNU Multiple Precision Arithmetic Library — Wikipedia, The Free Encyclopedia*. https://en.wikipedia.org/wiki/GNU_Multiple_Precision_Arithmetic_Library. [Dostęp: 27.04.2026]. 2026.

[2] Richard P. Brent and Paul Zimmermann. *Modern Computer Arithmetic*. Vol. 18. Cambridge Monographs on Applied and Computational Mathematics. Cambridge: Cambridge University Press, 2010. ISBN: 978-0-521-19469-3.

[3] Torbjörn Granlund and the GMP development team. *GNU MP: The GNU Multiple Precision Arithmetic Library*. 6.3.0. Dostęp online. 2023. URL: https://gmplib.org/manual/.

[4] Makoto Matsumoto and Takuji Nishimura. "Mersenne Twister: A 623-Dimensionally Equidistributed Uniform Pseudorandom Number Generator". In: *ACM Transactions on Modeling and Computer Simulation* 8.1 (1998), pp. 3–30. DOI: 10.1145/272991.272995.

[5] Python Software Foundation. *Integer Objects*. Online; accessed 2026. Python Software Foundation. URL: https://docs.python.org/3/c-api/long.html.

[6] Python Software Foundation. *random — Generate pseudo-random numbers*. Online; accessed 2026. Python Software Foundation. 2024. URL: https://docs.python.org/3/library/random.html.